인터폰 상태감지 모듈 — 태블릿 호스트 앱 개발자 전달자료

항목	내용
문서 일자	2026-10-19
대상 독자	Android 태블릿에서 상태감지 모듈의 호스트 애플리케이션을 개발하는 엔지니어
프로토콜 버전	통신_프로토콜_정리.md v1.5 (16바이트 ASCII 응답)
장치	인터폰 상태감지 모듈 (STM32F103 메인 MCU, USB-CDC 가상 시리얼)
검증 상태	장치 펌웨어와 Windows 호스트 프로그램은 실기 동작 확인 완료. 동봉된 Android 참고 소스는 **컴파일/실기 시험을 하지 않았음**

---

1. 한눈에 보기

- 태블릿이 USB **호스트**, 장치가 CDC(가상 COM) 디바이스다. 태블릿 앱은 **100ms마다 4바이트 요청**을 보내고 **16바이트 ASCII 응답**을 받는다.
- 응답에는 장치가 판정한 **인터폰 상태**(대기/호출/통화/전원 이상/통신 이상)와 **Tx, Rx(Power 대비 %)**, **Power 전압**이 들어 있다. 앱은 기본적으로 장치가 판정한 상태를 그대로 표시한다.
- 장치는 요청에만 응답한다. 상태가 바뀌어도 먼저 보내지 않는다.
- **앱이 10초 이상 요청을 보내지 않으면 장치가 USB를 3초간 끊었다 다시 연결한다(반복).** 앱은 백그라운드에서도 폴링을 유지하거나, 끊김/재연결을 정상 시나리오로 처리해야 한다. (4절)

```
[인터폰 라인] -> 프론트엔드 MCU -> 절연 -> 메인 MCU(STM32F103) -- USB-CDC --> [태블릿 앱]
                                                               요청 0D 52 0D 0A (100ms)
                                                               <-- 응답 16바이트 ASCII
```

---

2. USB 연결 정보

항목	값
장치 클래스	CDC-ACM (통신 인터페이스 + 데이터 인터페이스, 벌크 IN/OUT)
VID / PID	0x0483 / 0x5740 (십진수 1155 / 22336). **프로토타입용 ST 기본값이며 양산 시 변경될 수 있음**
제품 문자열	"STM32 Virtual ComPort"
속도	USB Full-Speed. CDC 보레이트 설정은 의미가 없다 (설정해도 무시됨. 115200 등으로 지정해도 무방)
흐름 제어	필요 없음. DTR/RTS 상태와 무관하게 동작한다 (참고 소스는 SET_CONTROL_LINE_STATE로 0x0003을 보내지만 필수는 아님)
전원	태블릿이 OTG 호스트로서 VBUS 5V를 공급해야 한다. 장치는 VBUS를 PA9로 감지하며, VBUS가 없으면 재연결 동작도 수행하지 않는다
케이블	데이터선이 살아 있는 OTG 케이블 필수. **충전 전용 케이블이면 충전만 되고 장치가 인식되지 않는다** (개발 중 실제로 발생)

구현 방법은 두 가지가 가능하다.

1. Android USB Host API(`UsbManager`, `UsbDeviceConnection.bulkTransfer`)로 CDC Data 인터페이스의 벌크 IN/OUT 엔드포인트를 직접 사용 (동봉된 참고 소스 방식, 외부 라이브러리 없음)
2. usb-serial-for-android 같은 CDC-ACM 라이브러리 사용 (포트 읽기 타임아웃 설정이 편리함)

어느 쪽이든 요청-응답이 짧은 고정 길이라 구조는 단순하다.

2.1 Android 설정 요점

- `AndroidManifest.xml`: `uses-feature android.hardware.usb.host`, `USB_DEVICE_ATTACHED` 인텐트 필터 + `device_filter.xml` (vendor-id=1155, product-id=22336)
- 장치 연결 시 사용자 권한 요청(`UsbManager.requestPermission`)이 필요하다. 인텐트 필터로 기동한 경우 "이 장치에 항상 사용" 선택으로 반복 권한 요청을 줄일 수 있다.
- 장치가 끊겼다 재연결되면 `ACTION_USB_DEVICE_DETACHED` 후 `ACTION_USB_DEVICE_ATTACHED`가 연속으로 온다. 이때 권한/연결 핸들을 다시 얻어 폴링을 재개한다.

---

3. 프로토콜

3.1 요청 (태블릿 -> 장치, 4바이트)

순서	값	의미
1	0x0D	STX
2	0x52 ('R')	상태 요청
3	0x0D	ETX1
4	0x0A	ETX2

```
0D 52 0D 0A
```

정의되지 않은 CMD는 응답하지 않는다. 요청에 별도 데이터 필드는 없다.

3.2 응답 (장치 -> 태블릿, 16바이트 고정, 전부 ASCII)

바이트	필드	내용
0	STX	0x0D
1	상태	`'0'` 대기 / `'1'` 호출 / `'2'` 통화 / `'3'` 전원 이상 / `'4'` 프론트엔드 통신 이상
2~5	Tx	4자리 10진수, 0.1% 단위 (`"0000"` ~ `"1000"`)
6~9	Rx	4자리 10진수, 0.1% 단위 (`"0000"` ~ `"1000"`)
10~13	Power	4자리 10진수, 전압 x 10 (0.1V 단위, `"0000"` ~ `"0263"`)
14	ETX1	0x0D
15	ETX2	0x0A

- **Tx, Rx**는 해당 채널 ADC를 Power ADC로 나눈 비율이다. 즉 Power를 100%(=1000)로 한 상대값이다. 예) `"0994"` = 99.4%. 라인 전압 추정은 `Power(V) x 필드값 / 1000`.
- **Power**는 `"0132"` = 13.2V. 마지막 자리는 ADC 오차로 +-1 정도 흔들릴 수 있다.
- 숫자는 높은 자리부터 쓴 일반 표기이며, 데이터 문자는 `'0'` ~ `'9'` 뿐이라 0x0D/0x0A와 겹치지 않는다.
- SUM(체크섬) 필드는 없다. USB 전송 자체의 CRC로 무결성을 확보한다.

3.3 상태별 필드 규칙

상태	의미	Tx / Rx	Power
`'0'` 대기	정상, 통화/호출 없음. 부팅 직후 첫 상태	실측 비율	실측값
`'1'` 호출	도어스테이션(외부)이 호출 중	실측 비율	실측값
`'2'` 통화	통화 중	실측 비율	실측값
`'3'` 전원 이상	인터폰 라인 전원이 약 1.3V 미만 (Power ADC < 200)	`"0000"`	실측값
`'4'` 통신 이상	모듈 내부 프론트엔드가 500ms 이상 데이터를 못 보냄 (고장, 케이블 이탈 등)	`"0000"`	`"0000"`

3.4 프레임 예시

상황	문자열	바이트 (hex)
대기 (Tx 3.2%, Rx 93.1%, 13.2V)	`<CR>0 0032 0931 0132 <CR><LF>`	`0D 30 30 30 33 32 30 39 33 31 30 31 33 32 0D 0A`
호출 (Tx 0%, Rx 0%, 13.2V)	`<CR>1 0000 0000 0132 <CR><LF>`	`0D 31 30 30 30 30 30 30 30 30 30 31 33 32 0D 0A`
통화 (Tx 99.4%, Rx 6.0%, 13.2V)	`<CR>2 0994 0060 0132 <CR><LF>`	`0D 32 30 39 39 34 30 30 36 30 30 31 33 32 0D 0A`
전원 이상 (0.6V)	`<CR>3 0000 0000 0006 <CR><LF>`	`0D 33 30 30 30 30 30 30 30 30 30 30 30 36 0D 0A`
통신 이상	`<CR>4 0000 0000 0000 <CR><LF>`	`0D 34 30 30 30 30 30 30 30 30 30 30 30 30 0D 0A`

(위 5개 프레임은 Windows 호스트의 `protocol.py` 파서로 검증했다.)

---

4. 폴링 / 타임아웃 / 재연결 (중요)

항목	규칙
폴링 주기	**100ms 이하** 권장. 상태 변화를 앱이 아는 시점은 폴링 주기에 따른다
응답 대기	요청 후 응답 16바이트를 수신할 때까지 300ms 정도 대기 (장치는 즉시 응답)
요청 전	이전 응답의 잔여 바이트가 남지 않도록 수신 버퍼를 비우고 요청을 보내면 안전하다
10초 무요청	장치가 USB D+ 풀업을 3초간 해제(연결 끊김) 후 다시 연결한다. 호스트가 계속 요청하지 않으면 13초 주기로 **반복**

USB 케이블 없음	장치는 위 재연결 동작을 하지 않는다
앱 측 복구	연속 5회 정도 응답 실패 시 연결을 닫고 다시 여는 것을 권장 (Windows 앱은 5회 실패 시 자동 재오픈)

설계 시 반드시 고려할 점

- 앱이 화면 꺼짐, 백그라운드 전환, 절전 등으로 10초 이상 요청을 못 보내면 장치가 USB를 끊었다 붙이므로, Android는 DETACHED/ATTACHED를 연속으로 받는다. 상태 감시 용도라면 **포그라운드 서비스(Foreground Service)로 폴링을 유지**하거나, 재연결을 자동 복구하도록 구현한다.
- 재연결 중(3초간)에는 데이터가 없으므로 UI에는 "통신 끊김/재연결 중" 상태를 별도로 표시한다. 이것은 장치 상태 `'4'`(프론트엔드 통신 이상)와는 다르다.
- 앱이 시작되면 첫 요청부터 즉시 응답이 온다. 부팅 직후 장치가 자발적으로 보내는 데이터는 없다.

---

5. 응답 파싱 (권장 절차)

1. 수신한 데이터를 **정확히 16바이트** 단위로 처리한다. (벌크 전송이 2회로 나뉘어 도착할 수 있으므로 16바이트가 모일 때까지 읽거나 타임아웃 처리한다)
2. 검증: `b[0]==0x0D`, `b[14]==0x0D`, `b[15]==0x0A`, 상태 문자가 `'0'`~`'4'`, Tx/Rx/Power 각 4문자가 모두 `'0'`~`'9'`, Tx/Rx <= 1000. 하나라도 어긋나면 해당 프레임을 폐기하고 오류로 센다.
3. 변환: `tx% = int(필드)/10`, `power(V) = int(필드)/10`, `라인전압 = power * 필드/1000`.

```kotlin
data class Frame(val state: Int, val txPm: Int, val rxPm: Int, val powerDv: Int)

fun parse(b: ByteArray): Frame? {
    if (b.size != 16) return null
    if (b[0] != 0x0D.toByte() || b[14] != 0x0D.toByte() || b[15] != 0x0A.toByte()) return null
    val st = b[1] - 0x30
    if (st !in 0..4) return null
    fun dec4(o: Int): Int? {
        var v = 0
        for (i in o until o + 4) {
            val d = b[i] - 0x30
            if (d !in 0..9) return null
            v = v * 10 + d
        }
        return v
    }
    val tx = dec4(2) ?: return null
    val rx = dec4(6) ?: return null
    val pw = dec4(10) ?: return null
    if (tx > 1000 || rx > 1000) return null
    return Frame(st, tx, rx, pw)
}
```

5.1 파서 시험 벡터 (비정상 프레임, 모두 폐기되어야 함)

케이스	입력	기대
길이 부족	대기 프레임의 앞 15바이트	폐기
STX 오류	첫 바이트 0x0E	폐기
상태 문자 오류	상태 `'5'`	폐기
숫자 오류	Tx 필드에 `'A'` 포함	폐기
범위 오류	Tx = `"1001"`	폐기

---

6. 화면 표시 권장

상태	표시 예	비고
대기	녹색 "대기"	
호출	황색 "호출 중" (강조)	엘리베이터 갇힘 호출. 알림/경보 연동 대상
통화	청색 "통화 중"	
전원 이상	적색 "인터폰 전원 이상"	Power 전압 함께 표시
통신 이상	적색 "모듈 내부 통신 이상"	장치 점검 필요
(앱 자체) 연결 끊김	회색 "장치 연결 안 됨 / 재연결 중"	장치 상태가 아님 (4절)

- Tx, Rx는 막대(%), Power는 전압(V)으로 표시하고, 진단용으로 원시 프레임(hex)을 로그로 남기면 현장 분석에 유용하다.
- 호출 상태는 놓치지 않도록 진동/소리/푸시 알림 등을 연동한다 (요구사항에 맞게 결정).

---

7. 호스트 재판정 (선택 기능)

일반적으로 장치가 보낸 상태(`'0'` ~ `'4'`)를 그대로 사용한다. 다음과 같은 경우에는 같은 프레임의 Tx/Rx/Power로 앱이 직접 판정할 수 있다.

- 현장 조건이 달라 기준값을 바꿔보고 싶을 때 (예: 24V 시스템)
- 장치 판정과 비교하며 진단할 때

7.1 입력과 기준값 (기본값, 모두 Power 대비 비율)

이름	조건	기본 기준 (0.1% 단위 필드값)
TxH	Tx > 750 (아니면 TxL)	75.0%
RxL	Rx < 380	38.0%
RxH	Rx > 750	75.0%
지정시간	조건이 연속으로 참이어야 하는 시간	400ms

7.2 상태 천이

현재	다음	조건	지정시간
대기	통화	TxH 그리고 Tx > Rx	400ms
대기	호출	TxL 그리고 RxL	400ms
호출	통화	TxH 그리고 Tx > Rx	400ms
호출	대기	Rx > Tx 그리고 RxH	즉시
통화	대기	Rx > Tx 그리고 TxL (RxL이면 제외)	즉시
통화	호출	TxL 그리고 RxL	400ms

- 어느 조건도 아니면 현재 상태를 유지한다. 조건이 겹치면 **RxL(호출)이 "Rx > Tx(대기)"보다 우선**한다 (통화 상태에서는 TxL 그리고 RxL을 먼저 검사).
- "지정시간"은 조건이 참이 된 시점부터 중간에 거짓이 되지 않고 계속 참이어야 하며, 거짓이 되면 타이머를 초기화한다.
- 장치가 `'3'`(전원 이상) 또는 `'4'`(통신 이상)를 보내면 재판정하지 않고 그 상태를 그대로 따른다 (Tx/Rx가 0이라 값으로 판정 불가). 복구되면 대기에서 다시 시작한다.
- 호스트가 Power 필드로 전원 이상을 직접 판정하려면 Power < 1.3V(필드값 13 미만)를 기준으로 한다.

7.3 호스트 판정 검증 시나리오 (50ms 간격 입력 기준, Python 구현으로 산출)

시나리오	입력 순서	기대 결과
호출 없이 수화기 듦	대기 값(Tx 3.2%, Rx 93.1%) 4프레임, 이후 Tx 99.4% / Rx 88.2% 지속	Tx가 높아진 시점부터 400ms 뒤 `통화`
호출 -> 통화 -> 대기	대기 2프레임 -> Tx 0% / Rx 0% 12프레임 -> Tx 99.4% / Rx 6.0% 12프레임 -> Tx 64% / Rx 93.1%	`호출` (0%가 시작된 지 400ms 뒤) -> `통화` (Tx 상승 후 400ms 뒤) -> `대기` (Tx 64%/Rx 93.1% 입력 즉시)
통화 종료 직후 오검출 방지	통화 중에서 Tx 64% / Rx 0%가 150ms -> Tx 64% / Rx 50% -> Tx 50% / Rx 93.1%	통화 유지 후 `대기`로 복귀, **중간에 `호출`이 표시되지 않음**

주의: 호스트 판정은 100ms 폴링 값으로 계산하므로 장치(50ms 프레임)보다 천이가 최대 100ms쯤 늦거나, 50ms 정도의 짧은 글리치는 보이지 않을 수 있다.

---

8. 예외 상황 정리

증상	원인	앱 대응
장치가 목록에 없음 / 권한 요청 불가	충전 전용 케이블, OTG 미지원, VBUS 미공급	사용자에게 케이블/OTG 확인 안내
응답 0바이트	잘못된 장치에 연결, 장치 재연결 중(3초), 장치 리셋 중	재시도 후 연속 실패 시 연결 재설정
응답 길이 오류	수신이 분할됨, 이전 응답 잔여 바이트	버퍼 비우고 재요청 (연속 시 재연결)
상태 `'4'`	모듈 내부 UART 링크 끊김	경고 표시, 장치 점검 안내

상태 `'4'`	모듈 내부 UART 응답 없음	경고 표시, 장치 점검 안내
상태 `'3'`	인터폰 라인 전원 이상	경고 표시, Power 전압 함께 표시
장치 분리 이벤트	케이블 분리 또는 10초 무요청 재연결	3~10초간 자동 재연결을 기다린 뒤 실패 시 사용자 알림

---

9. 알려진 제한 / 미결 사항

- **VID/PID는 프로토타입 값**이다. 양산 VID/PID가 확정되면 앱의 장치 필터(`device_filter.xml`, 코드 내 상수)를 함께 수정한다.
- **24V 현장은 실측하지 못했다.** 판정 기준은 Power 대비 비율이라 이론상 대응하지만, 통화 종료 직후 Tx 변화가 24V에서도 같은 크기이면 대기 복귀가 지연될 수 있다. 24V 현장 투입 시 장치 판정(`'0'`~`'2'`)과 호스트 재판정(기준값 변경)을 비교하여 확인한다.
- 호출 시 Rx가 약 0V, 통화 시 Tx가 약 99%라는 값은 COMMAX 4선식 인터폰의 실측(Power 13.25V)에 근거한다. 다른 제조사 인터폰에는 그대로 적용되지 않을 수 있다.
- 전원 이상/통신 이상 시 장치 LED 패턴은 임시 공용값이며 앱과는 무관하다.
- 동봉된 Android 참고 소스는 컴파일과 실기 시험을 하지 않았다. 구조와 파싱 로직의 참고용으로 사용한다.

---

10. 인수 시험 체크리스트

번호	시험 항목	확인
1	장치 연결 시 앱 자동 실행 및 USB 권한 처리	[ ]
2	100ms 폴링으로 Tx/Rx/Power, 상태가 갱신됨	[ ]
3	대기 -> 호출 -> 통화 -> 대기 시나리오에서 상태가 올바르게 표시됨 (실제 인터폰 조작)	[ ]
4	비정상 프레임(5.1절) 폐기 및 연속 오류 시 재연결	[ ]
5	USB 케이블 분리 후 재연결 시 자동 복구	[ ]
6	앱을 10초 이상 정지(백그라운드)한 뒤 복귀 시 장치 재연결 처리	[ ]
7	상태 `'3'`, `'4'` 표시 (모의 시험: 프론트엔드 연결 해제 시 `'4'`)	[ ]
8	호스트 재판정 값과 장치 판정값이 7.3절 시나리오와 일치	[ ]

---

11. 첨부 자료 (함께 전달 권장)

파일	내용
통신_프로토콜_정리.md	프로토콜 전체 정의 (v1.5), 상태 판정 상세
인터폰_상태감지_개발사양서.md	시스템 사양, 실측 신호 특성
`windows_monitor/protocol.py`	파서와 호스트 판정(`HostJudge`) 참고 구현 (Python, 시험 통과)
`IntercomMonitor` (Android 프로젝트)	`Protocol.kt`(파서, `HostJudge`), `MainActivity.kt`(USB Host API 직접 사용), `device_filter.xml`